

# ARCHITECTURE ET DÉPLOIEMENT MATÉRIEL

*Manuel d'installation complet du système d'acquisition et de traitement*

**Auteur :** Samuel Bassetto, LABAC, École Polytechnique Montréal

**Date :** 8 juin 2026

**Projet :** HML (Helmet/Head Mounted LiDAR)

## 1. Configuration Hôte du Raspberry Pi 4

Le déploiement du système HML nécessite une installation propre d'un système d'exploitation Linux 64-bit sur le Raspberry Pi 4, de préférence la distribution Ubuntu Server ou Raspberry Pi OS Lite. Cette architecture logicielle en version 64-bit est indispensable pour exploiter pleinement les registres d'adresses de la mémoire vive lors des calculs massifs d'odométrie inhérents au traitement des données géométriques.

Une fois le système flashé sur la carte SD, l'accès initial s'effectue en reliant le matériel à un écran ou via une console série afin d'effectuer les mises à jour de sécurité indispensables. L'activation des interfaces matérielles requiert l'installation des dépendances système nécessaires à la communication avec les broches d'entrées et sorties physiques, opérations exécutées par le biais des gestionnaires de paquets natifs de la distribution.

```
sudo apt update && sudo apt upgrade -y  
sudo apt install -y python3-pip python3-dev g++ git
```

## 2. Déploiement de l'Infrastructure Docker

La conteneurisation des bibliothèques de traitement garantit une isolation parfaite du système et simplifie le transfert des modules entre les différentes plateformes de calcul. L'installation du moteur Docker s'effectue par le biais du script automatisé officiel disponible en ligne, ce qui permet d'éviter les conflits de versions liés aux dépôts tiers.

Après l'installation du service, il est indispensable d'ajouter l'utilisateur local au groupe système dédié afin de pouvoir exécuter les commandes d'administration sans exiger l'élévation systématique des privilèges par la commande `sudo`. La configuration du démon de conteneurisation se termine par l'activation du lancement automatique du service à chaque initialisation matérielle du nano-ordinateur.

```
curl -fsSL https://get.docker.com -o get-docker.sh  
sudo sh get-docker.sh  
sudo usermod -aG docker $USER  
sudo systemctl enable docker && sudo systemctl start docker
```

Pour assurer la persistance des données récoltées lors des phases d'exploration sur le terrain, la création d'un espace d'échange partagé appelé volume de données est requise. Ce volume fait le pont entre le stockage physique du Raspberry Pi et l'arborescence interne du conteneur, permettant aux fichiers

d'enregistrement ROS bag de subsister de manière définitive même en cas de destruction ou de mise à jour complète de l'image logicielle sous-jacente.

```
docker volume create hml_data_volume
```

### 3. Configuration du Pont SSH via Hotspot Cellulaire

Les contraintes opérationnelles du milieu souterrain imposent l'établissement d'une liaison réseau directe dénuée d'équipement de routage tiers. La solution consiste à exploiter le partage de connexion cellulaire d'un smartphone pour créer un réseau local de terrain où le MacBook de contrôle et le Raspberry Pi peuvent coexister de manière transparente.

Pour éliminer les coupures intempestives et assurer la compatibilité matérielle avec le récepteur sans fil intégré du Raspberry Pi 4, le point d'accès de l'iPhone doit être configuré avec l'option de maximisation de la compatibilité activée, ce qui force l'émission sur la bande stable de 2,4 GHz. L'inscription des identifiants du réseau s'effectue au niveau du gestionnaire réseau de l'hôte, en veillant à spécifier l'autorité de certification adéquate.

```
sudo nmcli device wifi rescan
sudo nmcli device wifi connect "Nom_iPhone" password "Mot_De_Passe"
```

En configuration de terrain dénuée d'écran, la découverte de l'adresse IP dynamique attribuée au Raspberry Pi par le smartphone s'effectue depuis le terminal du MacBook Air en interrogeant la table de correspondance de résolution d'adresses locales par le biais de la commande arp ou via le protocole de diffusion mDNS.

```
arp -a | grep -i "raspberrypi"
ping raspberrypi.local
```

### 4. Spécifications du Conteneur et Dépendances Matérielles

Le conteneur applicatif principal embarque la distribution ROS 2 Humble pour centraliser la réception des trames du télémètre laser. Le pilotage des périphériques électroniques secondaires connectés sur les broches physiques s'appuie sur l'intégration de la bibliothèque RPi.GPIO, dont la configuration est adaptée pour s'exécuter à l'intérieur de l'espace virtuel isolé de Docker.

L'instanciation du conteneur requiert l'injection de paramètres spécifiques afin d'autoriser l'accès aux interfaces matérielles et réseau de l'hôte. La commande d'initialisation doit inclure le partage du réseau hôte, l'accès privilégié aux périphériques système et le montage croisé des répertoires de stockage pour l'arborescence des résultats. Le chemin d'accès local a été mis à jour pour s'adapter à la variable de session d'un système Unix standard.

```
docker run -it --privileged --network host \
-v /Users/${USER}/Desktop/Expedition_Data/ros2_ws/results:/root/ros2_ws/results \
--name hml_m4 ros2_humble_livox:latest bash
```

## 5. Requirements et Modules Logiciels Installés

Le bon fonctionnement de la chaîne algorithmique repose sur l'installation de dépendances scientifiques et de bibliothèques spécialisées dans la manipulation de structures spatiales complexes. Ces paquets logiciels gèrent aussi bien l'analyse statistique que la lecture séquentielle des bases de données SQLite3 générées par l'enregistreur de données.

L'environnement d'exécution requiert l'installation conjointe de modules de calcul matriciel et de structures d'indexation géométrique performantes pour accélérer le traitement post-expédition. Les exigences logicielles strictes incluent l'implémentation de filtres topographiques basés sur des projections orthogonales et la détection des structures par partitionnement de l'espace.

```
pip3 install numpy scipy rosbags numpy-ros python-dotenv
```

La bibliothèque `scipy` fournit l'infrastructure essentielle `cKDTree` utilisée pour optimiser les recherches de voisinage tridimensionnel à grande échelle. Ce module évite l'effondrement des performances du processeur face aux millions de points capturés en limitant le calcul aux enveloppes locales. La bibliothèque `rosbags` permet quant à elle l'analyse et la manipulation directe des fichiers d'enregistrement sans exiger l'exécution complète d'une instance active de ROS 2.

## 6. Validation et Traitement Spéléologique

La validation du pipeline se traduit par l'exécution des modules Python de post-traitement sur les fichiers bruts. Le premier module extrait le nuage de points figé dans le référentiel global et l'exporte au format Point Cloud Data pour éliminer la composante temporelle. Le second module traite la trajectoire issue de l'odométrie directe, fragmente le déplacement selon un intervalle régulier de un mètre cinquante pour poser les stations, et applique une coupe transversale vectorielle perpendiculaire à l'axe d'avancement.

Cette approche par tranches isole les parois afin d'extraire les dimensions exactes de la galerie à gauche, à droite, en haut et en bas, tout en générant les visées radiantes secondaires pour l'habillage tridimensionnel. Le fichier généré respecte strictement la structure sémantique de la norme Visual Topo, permettant une intégration immédiate et une modélisation fidèle de la structure géologique explorée.